‘Hardware Hustlers’ Mathematical Accelerator Report

Authors

Clyde Pangilinan 02381828

Ali Rabie 02372729

Alexander Brown 02412227

Kevin Huang 02461751

Jack Hollway 02399831

Marker

Sarim Baig

June 19, 2025

Abstract

This report details the design and implementation of a hardware accelerator for generating the Mandelbrot set on an FPGA. The project requirements were given as follows:

- Develop an accelerator for the Pynq system-on-chip FPGA platform, written in Verilog or SystemVerilog, that can generate a video output visualising the chosen function
- Identify opportunities for parallelism to increase calculation throughput
- Use numerical analysis to determine the optimum number representations and bit-widths to achieve a suitable accuracy
- Use human-computer interaction to allow a user to manipulate and understand the visualisation
- Quantify the impact of design implementation decisions in terms of algorithmic throughput

The solution utilises fixed-point arithmetic, with a 32-bit word length providing sufficient precision for a wide range of zoom levels whilst optimising hardware resource utilisation. Important design decisions made include constraining certain values (e.g. zoom and maximum iterations) to be powers of 2 so their associated multiplications and divisions can be replaced by bit shifts, and optimising DSP slice utilisation for squaring operations resulting in a decrease from 12 to 10 DSP slices per engine.

Analysis of the performance demonstrates a linear speed-up with parallel engines, alongside a constant offset which was attributed to sequential bottlenecks. This project successfully demonstrates FPGA's capabilities in accelerating complex computations.

GitHub Repo.

Contents

1 Introduction

1.1 General Introduction

This report details the group's Second Year Electronics Design Project, in which it was decided to design a Mathematics Accelerator. The aim of this project was to create an interactive educational tool which visually represents the Mandelbrot set utilising the PYNQ-Z1 SoC FPGA development board in real time with an interactive user interface.

1.2 Mandelbrot Set

The Mandelbrot set is a set of complex numbers c where the sequence $z_n = z_{n-1}^2 + c$ does not diverge as $n \to \infty$ for $z_0 = 0$. By applying this formula across the complex plane, we get the fractal shown below with an infinitely complex boundary. The set is coloured depending on how quickly a represented complex number escapes toward infinity, with the colour black indicating that it stays bounded in absolute value. This was chosen for several key reasons: each pixel can be computed independently which allows for embarrassingly parallel computation, the infinitely complex boundary means that the FPGA's computational power can be showcased, and it is a visually interesting fractal that is ideal to demonstrate and interact with as an educational tool. In this project the infinitely complex and beautiful structures are demonstrated and explained in detail.

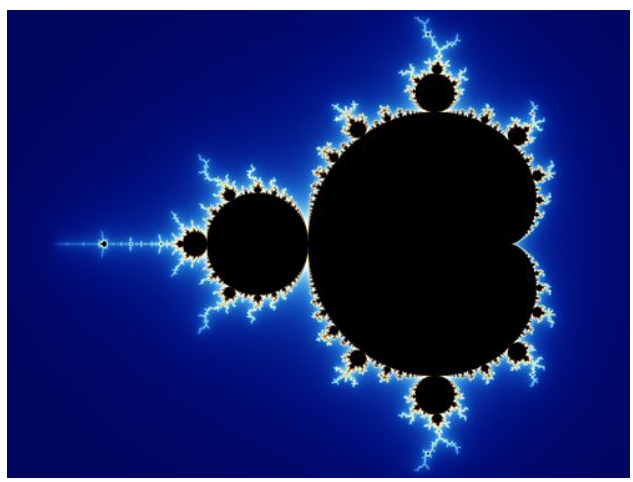


Figure 1: Example Mandelbrot set render[1]

2 Project Management

2.1 Project Objectives

To consolidate the aims of the project, in depth research was conducted by all team members and presented in a group meeting to reach a decision. Once a general plan was agreed on, detailed project specifications were defined and are shown in Table 1:

General Requirement	**Project Specific Requirement**
The system shall display a computationally intensive visualisation of a mathematical function that is computed in real time and embarassingly parallel	The system will display the mandelbrot set computed in real time via the FPGA for use as an interactive tutorial for educational purposes
The visualisation shall be generated with the supplied PYNQ-Z1 SoC FPGA development board with an accelerator for the soft logic of the FPGA that computes the inner loops of the calculation and exceed the performance of a software equivalent	The optimised hardware accelerator will exceed the performance of the software equivalent embedded within a webpage with use of benchmarking to make a comparison
The system shall provide a user interface to enhance the function of the visualisation as an educational tool	The system will be adjustable for use as an interactive tutorial via an intuitive and easy to use user interface

Table 1: Project Specific Requirements

2.2 Project Framework

After in-depth research on deciding project specifications, detailed planning and organisation was required to ensure that the team was on track to achieve individual and team goals. Team meetings were held informally daily and formally weekly with an agile approach adopted, containing four separate sprints as seen in Table 2:

Sprint	**Objectives**
Sprint 1: 19/5/25 - 25/5/25	• Assign team roles and set up Software • Design single engine Mandelbrot • Design software simulation
Sprint 2: 26/5/25 - 01/6/25	• Enable panning and zooming of Mandelbrot display • Design prototype webpage • Interface single engine Mandelbrot
Sprint 3: 2/6/25 - 8/6/25	• Interim presentation • Display Mandelbrot set via HDMI • Design user interface
Sprint 4: 9/6/25 - 19/6/25	• Complete integration of user interface and webpage • Optimise and debug • Finish writing report • Interview and Demo

Table 2: Agile Approach Breakdown

Each sprint was broken down into project milestones, clearly indicating group progress. The final sprint is longer than the others to account for unforseen delays and allow for any integration issues.
A Gantt chart was designed to give an ongoing visual representation of the project framework, with formal group meetings, group consultations and important deadlines shown.

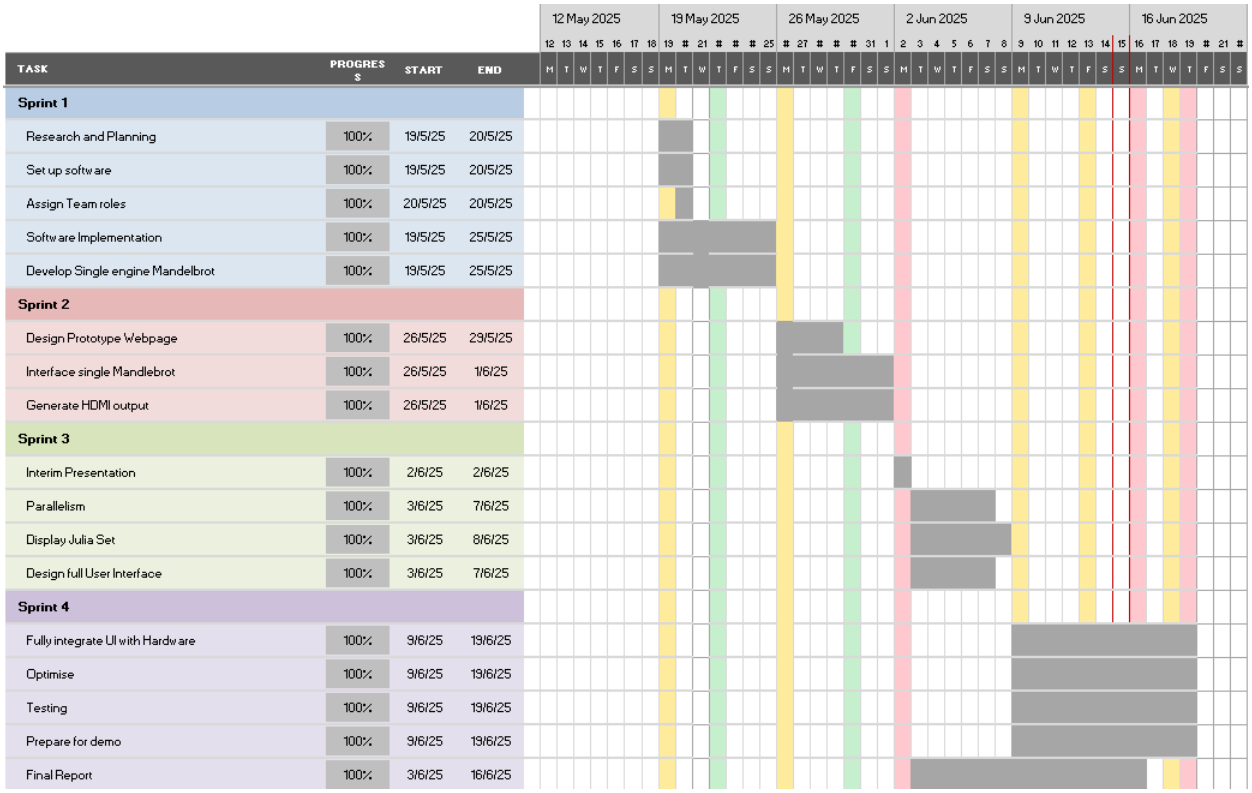


Figure 2: Project Gantt Chart

2.3 Individual Roles

After defining project specific requirements, each group member was assigned a specific role in the team, as shown below. Roles were allocated based on individual preference and experience.

Person	**Role**
Alex Brown	Hardware Design and Integration
Ali Rabie	Hardware Design and Integration
Clyde Pangilinan	Software Implementation and Testing, User Interface
Jack Hollway	Software Design, Testing and User Interface
Kevin Huang	Testing, Simulation, and Multiengine Integration

Table 3: Group Member Roles

Although clearly defined, these roles were flexible and allowed group members to contribute in all aspects of the project. Communication and file sharing was done via WhatsApp group and a shared GitHub repo, allowing for easy collaboration and daily meetings held in person in which we would work in collaboration.

3 Hardware implementation

3.1 Final Design Block Diagram

Figure 3 shows a high-level block diagram of our final design. A user makes inputs in the user interface which are then converted into visualisation parameters that are packed and sent to the Jupyter notebook server on the FPGA. The FPGA generates frames using our hardware which is sent to the user interface and displayed to the user.

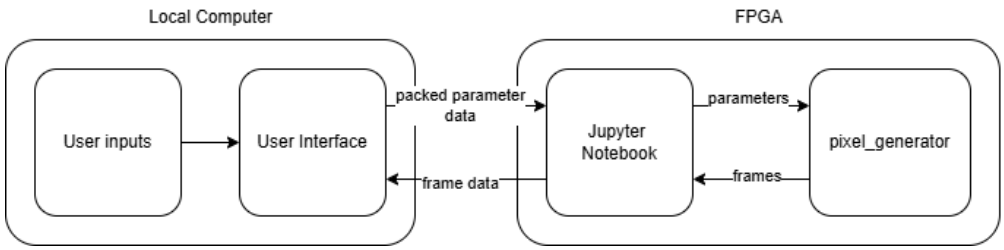


Figure 3: Block diagram of system

3.2 Simulation

The goal of developing a simulation environment for the image generator was threefold: to preview generated images without deploying to hardware, to estimate the number of clock cycles required to compute a frame, and to verify that the correct number of words are output for handshaking, all while avoiding the time overhead of repeatedly loading the design onto the FPGA.

3.2.1 Verilator

Verilator is a high-performance HDL simulation tool that supports integration with C++ testbenches. This capability enables more flexible and powerful simulation workflows compared to tools such as Icarus Verilog. In this project, Verilator was used to simulate the pixel generation logic, extract RGB values, and output the results to a `.csv` file. A Python script was then used to convert this file into a `.png` image for visual inspection.

3.2.2 Testbench

The provided Verilog testbench contained the core signal-driving logic and was adapted into a Verilator friendly top-level module. This top module exposed the RGB output signals from the `pixel_generator.v` module and received clock and reset signals driven by the Verilator-based C++ testbench.
The testbench captured pixel data and streamed it into a `.csv` file with the format: `x,y,r,g,b`. Each line in the file represents a pixel with its position and RGB colour value.

3.2.3 Python

To simplify post-processing and image generation, Python was chosen over C++ due to its ease of use and availability of relevant libraries. The `Pillow` library was used to read the `.csv` file and generate a corresponding `.png` image. Although performance was not a critical factor, the script was sufficiently fast for simulation purposes and provided a convenient way to visually validate the Mandelbrot rendering.

```
from PIL import Image
import os
import csv

# First, determine the width and height from the CSV
max_x = max_y = 0
with open("pixels.csv", newline='') as csvfile:
    reader = csv.DictReader(csvfile)
    for row in reader:
        x = int(row['x'])
        y = int(row['y'])
        max_x = max(max_x, x)
        max_y = max(max_y, y)

# Create an empty RGB image
image = Image.new('RGB', (width, height))
pixels = image.load()

# Fill the image with RGB values from the CSV
with open("pixels.csv", newline='') as csvfile:
    reader = csv.DictReader(csvfile)
    for row in reader:
        x = int(row['x'])
        y = int(row['y'])
        r = int(row['r'])
        g = int(row['g'])
        b = int(row['b'])
        pixels[x, y] = (r, g, b)
        if x == max_x and y == max_y:
            print("Reached the last pixel in the CSV.")
            break

# Save the image
output_path = "output_from_csv.png"
image.save("output_from_csv.png")
```

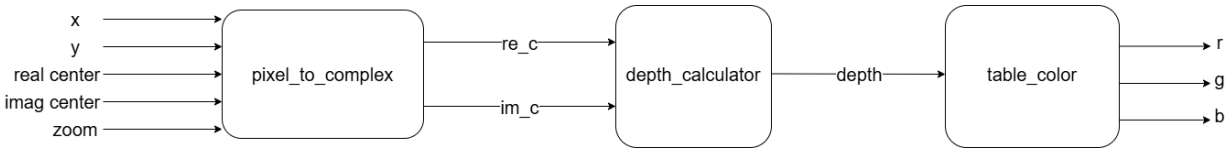
Listing 1: Image generation

3.3 Single Mandelbrot engine

The general procedure used to generate the image is described below:

- Each pixel within the image is mapped to a complex number using the `pixel_to_complex` module. This complex number is scaled by the zoom factor and center c, a complex number identifying the center of the image. These are inputs intended to be adjustable by the user, enabling them to navigate around the fractal.
- The complex number c associated with each pixel is input to the recursive function $z_{n+1} = z_n^2 + c$ and the depth is calculated as the number of iterations needed for that complex number to 'escape' (diverge to infinity).
- If the number does not escape within the 'maximum iterations' set by the user, it is part of the Mandelbrot set and the pixel is colored black. Otherwise, the pixel is colored depending on its depth.

The depth acts as an address input to a Look Up Table (LUT) where the corresponding colour RGB values are stored.



3.3.1 Pixel to complex

`pixel_to_complex.sv` was designed to map screen coordinates (x, y) to complex numbers for depth calculation. This module receives `zoom`, `real_center`, `imag_center`, `x` & `y` as inputs and outputs the corresponding complex number `real_part` & `im_part`.

Firstly, a single cartesian unit in the fixed point representation was defined as 1 << FRAC. This can be multiplied by 3 to obtain the intended range for width, and by 2 to obtain the intended range for height. This may then be divided by the `zoom` factor; increasing zoom would allow a smaller region of the fractal to be viewed in higher detail, so the range of width & height would be reduced.

The minimum real value may hence be calculated as the `real_center` $-\frac{1}{2}$`real_width` and the maximum imaginary value may be calculated as `imag_center` $-\frac{1}{2}$`imag_height`. The x and y pixel coordinates are scaled by the $\frac{\texttt{real\_width}}{\texttt{SCREEN\_WIDTH}}$ and $\frac{\texttt{imag\_height}}{\texttt{SCREEN\_HEIGHT}}$ respectively and this is used to determine the real and imaginary parts of the complex number associated with that pixel.

```
always_ff @(posedge clk) begin
    real_part <= real_min + x_scaled;
    im_part   <= imag_max - y_scaled;
end
```

Listing 2: Updating the complex number synchronously

3.3.2 Depth calculator

The `depth_calculator.sv` module was designed to calculate the depth associated with a given complex number (i.e. the number of iterations of the formula $z_{n+1} = z_n^2 + c$ needed for that number to diverge). The complex number may be split into the real and imaginary components:

$$\Re(z_{n+1}) = \Re(z_n)^2 - \Im(z_n)^2 + \Re(c) \text{ and } \Im(z_{n+1}) = 2\Re(z_n)\Im(z_n) + \Im(c).$$

The process of calculating each iteration and checking the escape condition ($|z_n|^2 \geq 4$) is split into multiple stages as it would otherwise incur too great of a combinational delay for one clock cycle leading to timing violations. The first `IDLE` state awaits the start signal to reset z and depth to begin calculation for a new complex number. The `ITER_1` state computes all of the partial products required to obtain $\Re(z_n)^2$, $\Im(z_n)^2$ and $2\Re(z_n)\Im(z_n)$ using the principle described in Section 3.3.4. The `ITER_2` state forms the full products from the partial products and `ITER_3` finds the next $\Re(z_n)$ and $\Im(z_n)$ from these products by applying the formula shown above.

The escape condition is then checked; if $\Re(z_n)^2 + \Im(z_n)^2 \geq 4$, it is known that the complex number would diverge and so the escape condition is met. Otherwise, the state machine would return to `ITER_1` and

begin another iteration. If the state machine reaches the maximum iterations set by the user (i.e. `depth == max_iter`) or if the escape condition is met earlier, the state machine would advance to the `FINISHED` state where the output signal `done` would be set high and `final_depth` is set to the depth obtained.

The `done` signal acts as an enable signal for the `table_color.sv` module which will be described below; it indicates that `final_depth` is the correct depth associated with that pixel to ensure that the colour output is correct for that pixel.

The state diagram for the `depth_calculator.sv` module is shown below.

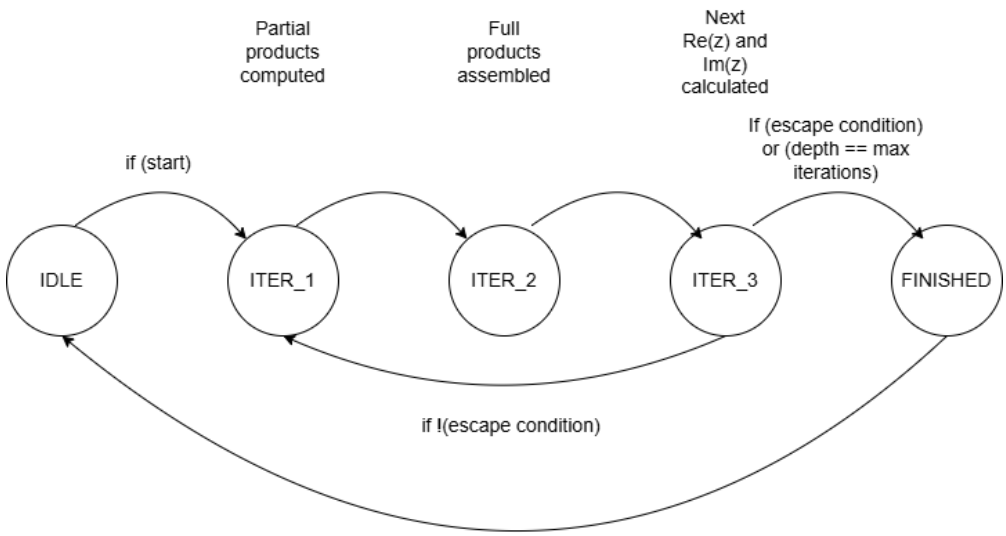


Figure 4: FSM for depth calculation

3.3.3 Table color

In order to colour the image, the `table_color.sv` module was developed. This is essentially a look-up table (LUT) that accepts `final_depth` as an input and outputs the corresponding RGB colour value. A Python script was written that generates a `.mem` file containing 1024 RGB colors which are loaded into memory in `logic [23:0] color_lut [0:1023]`.

If the `depth == max_iterations`, the pixel would not have escaped and so it would be colored black as it is considered a part of the Mandelbrot set. Otherwise, the depth is multiplied by 1023, then divided by `max_iterations` and truncated to 10 bits to find the address input to the LUT. This normalisation ensures that as `max_iterations` varies, the generated Mandelbrot set maintains a consistent colour scheme. The `valid` signal is asserted to interface with the `packer.v` module in `pixel_generator.v` to indicate that the pixel is ready.

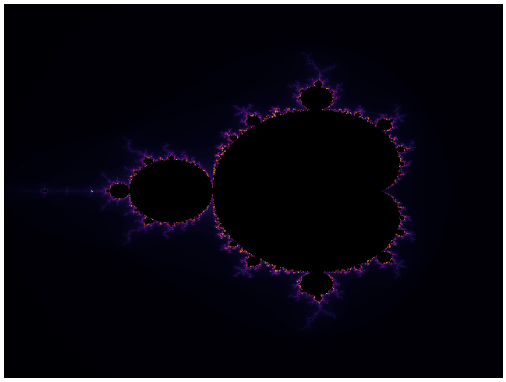


Figure 5: Example frame coloured using `table_color.sv`

```
        always_ff @(posedge clk) begin
        valid <= 1'b0; // Default to not valid
        if (en) begin
            valid <= 1'b1;
            if (!escape)
                color <= 24'h000000;
            else
                color <= color_lut[address];
        end
    end
```

Listing 3: Colour output

3.3.4 User interface

In order to allow the user to generate the fractal at different positions, the `regfile` instantiated in the `pixel_generator.v` module was utilised. These registers may be written to via the Jupyter notebook, and so interactions with the user interface may allow rendering of the Mandelbrot set with different, user-defined parameters such as zoom and center position. These inputs may then be taken as wires from these registers rather than being defined as `localparam`.

```
wire [31:0] MAX_ITER = regfile[0];
wire [31:0] ZOOM = regfile[1];
wire [31:0] REAL_CENTER = regfile[2];
wire [31:0] IMAG_CENTER = regfile[3];
```

Listing 4: Using regfile to store inputs

In the Jupyter notebook used to interface with the FPGA, the code shown below was added to a cell. The `pixgen` object acts as a software proxy for the `pixel_generator.v` module within the overlay, and it is used to allow the contents of `regfile` to be manipulated via the Jupyter notebook.

```
pixgen = overlay.pixel_generator_0

zoom_val = 1_000_000
max_iter_val = 512
real_center_val = int(-0.75 * (2**28)) #  -0.75 in Q4.28 format
imag_center_val = 0

# Write parameters to the registers
```

```
pixgen.register_map.gp0 = max_iter_val # MAX_ITER
pixgen.register_map.gp1 = zoom_val     # ZOOM
pixgen.register_map.gp2 = real_center_val # REAL_CENTER
pixgen.register_map.gp3 = imag_center_val # IMAG_CENTER
```

Listing 5: Using regfile to store inputs

3.3.5 Implementation & improvements

In simulation, the single engine produced high-quality images with aesthetically pleasing results (see Fig. 6). The colors of `table_color.sv` provided good contrast, allowing patterns of the fractal to be seen, and the engine captured details of the fractals effectively for higher zoom factors.

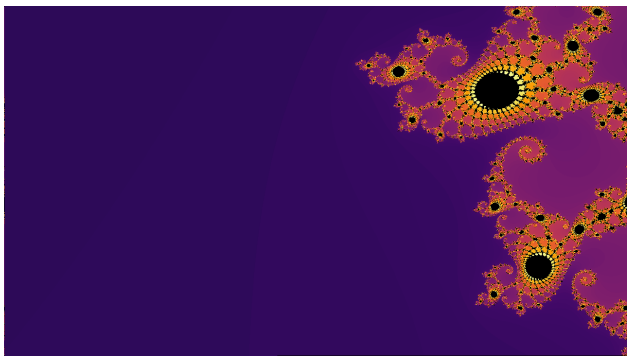


Figure 6: Single engine simulated frame for $\times 16384$ zoom

However, when implemented on the FPGA, the results did not match those in simulation. This was due to timing violations; although care was taken to ensure the depth calculator met timing, there were violations in the other modules like `pixel_to_complex.sv` and `table_color.sv`. This was due to operations like multiplications and divisions being performed in sequence within `always_comb` blocks to scale the numbers. These introduced critical paths longer than the clock period, leading to timing violations. Rather than splitting these calculations into stages to ensure timing is met and increasing the number of clock cycles required for each pixel, the logic was instead simplified.

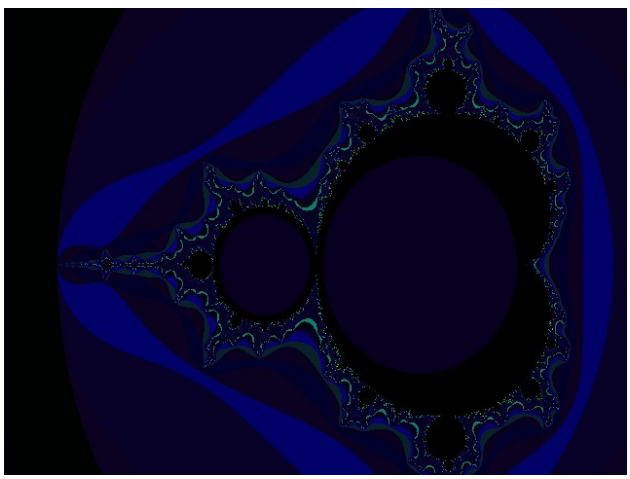


Figure 7: Single engine implementation generated frame

The zoom levels and maximum iterations are now restricted to powers of 2, allowing their multiplications and divisions to be replaced with bit shifts. Although this limits user flexibility, the significant gains in throughput and decreased resource usage render it a valuable compromise for high-performance image generation.

The zoom factor was redefined; instead of dividing by `ZOOM` in `pixel_to_complex.sv`, the range is shifted right by `ZOOM`. This is equivalent to dividing by 2^{ZOOM}. As the zoom factor is incremented, the image is

zoomed in by a factor of $\times 2$. Additionally, the logic behind the `pixel_to_complex.sv` module was reworked to be more efficient. Instead of calculating the scaled x and y values every clock cycles using computationally expensive multiplications, a 'real' and 'imaginary' step size is calculated in an `always_comb` block, meaning they are calculated only when the `ZOOM` input is changed.

As `SCREEN_WIDTH` and `SCREEN_HEIGHT` are compile-time constants, dividing `real_width` and `imag_height` by them is not problematic; this operation would be synthesised as a multiplication followed by a shift in Vivado if necessary, rather than implementing a full division circuit.

```
always_comb begin
     if (rst) begin
         step_real = 0;
         step_imag = 0;
         real_min  = 0;
         imag_max  = 0;
     end
     else begin
         real_width  = (3 * one_fp) >>> ZOOM;
         imag_height = (2 * one_fp) >>> ZOOM;

         step_real = real_width  / SCREEN_WIDTH;
         step_imag = imag_height / SCREEN_HEIGHT;

         real_min  = real_center - (real_width >>> 1);
         imag_max  = imag_center + (imag_height >>> 1);
     end
 end
```

The 'per-pixel' logic now stores the previous x and y in registers, and compares them with the current coordinates. If either of the (x, y) coordinates are incremented, the corresponding output `real_part` or `im_part` is incremented by `step_real` or `step_imag` respectively. This replaces the problematic multiplication and division operations which previously led to timing violations.

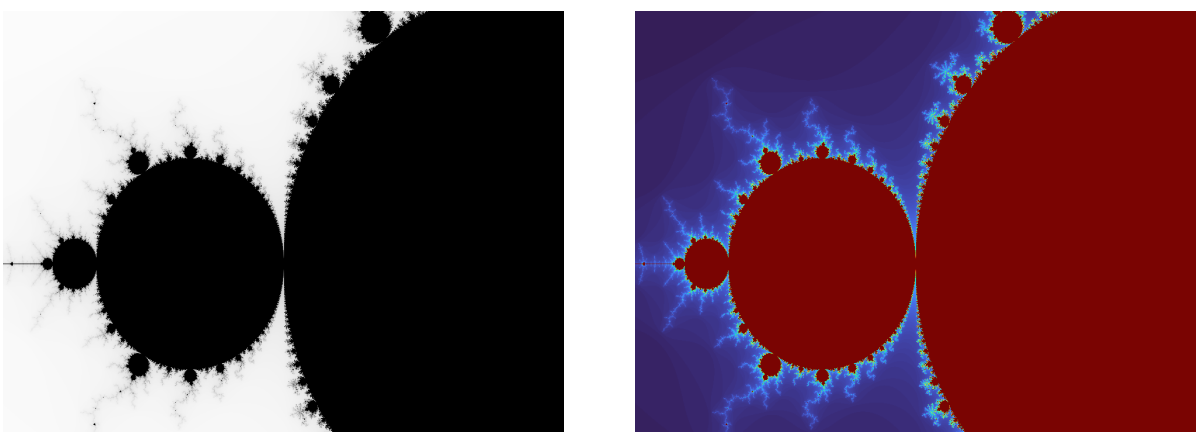
```
always_ff @(posedge clk) begin
      if (rst) begin
   // start at real_min, imag_max; the top left corner
         prev_x     <= 0;
         prev_y     <= 0;
         real_part  <= real_min;
         im_part<= imag_max;
     end else begin
         prev_x <= x;
         prev_y <= y;
         if (x == 0) begin
   // beginning of a new line
             real_part <= real_min;
         end
         else if (x != prev_x) begin
   // x has just incremented:
             real_part <= real_part + step_real;
         end
         if (y == 0) begin
   // beginning of a new frame:
             im_part <= imag_max;
         end
```

```
            else if (y != prev_y) begin
        // y has just incremented:
                im_part <= im_part - step_imag;
            end
        end
    end
```

Furthermore, the `table_color.sv` module, which previously performed colour lookup based on the depth associated with each pixel, was removed completely from the design and the image was generated in gray scale. The 'intensity' of each pixel was proportional to the depth associated with it. Using shifts, this completely bypasses the need for the complex operations needed to normalise address values and the large amounts of memory required to implement the `table_color.sv` module. Each pixel's intensity can be easily mapped to a colour in software, allowing greater flexibility in terms of colour schemes that may be applied.

An important benefit of performing the colour mapping in software is the ability to easily implement colour-blind friendly colour schemes to ensure the generated images are accessible and comfortable to view for a wider audience. This ensures our design is more inclusive.

```
    assign b = intensity;
    assign g = intensity;
    assign r = intensity;
    assign color = (final_depth == MAX_ITER) ? 255 :
                   ((MAX_ITER_LOG > 8) ? (final_depth >> (MAX_ITER_LOG - 8)) :
                                         (final_depth << (8 - MAX_ITER_LOG)));
    assign intensity = 255 - color; // Invert the color for visualization
```



(a) Grey-scale generated image (b) Example software colour scheme applied

Figure 8: Generated images after improvements

3.4 Numerical format

3.4.1 Fixed point vs floating point representation

Fixed-point representation was chosen over floating-point because this application benefits more from precision than range. Since the Mandelbrot set lies within a circle of radius 3, the extended range of floating-point numbers is unnecessary. In contrast, fixed-point allows for greater precision for a given word length, which is valuable for capturing fractal detail at higher zoom levels.

Additionally, fixed point calculations require fewer hardware resources than floating point which allows for more cores to be instantiated for a given hardware budget, improving parallelism as more pixels can be processed simultaneously.

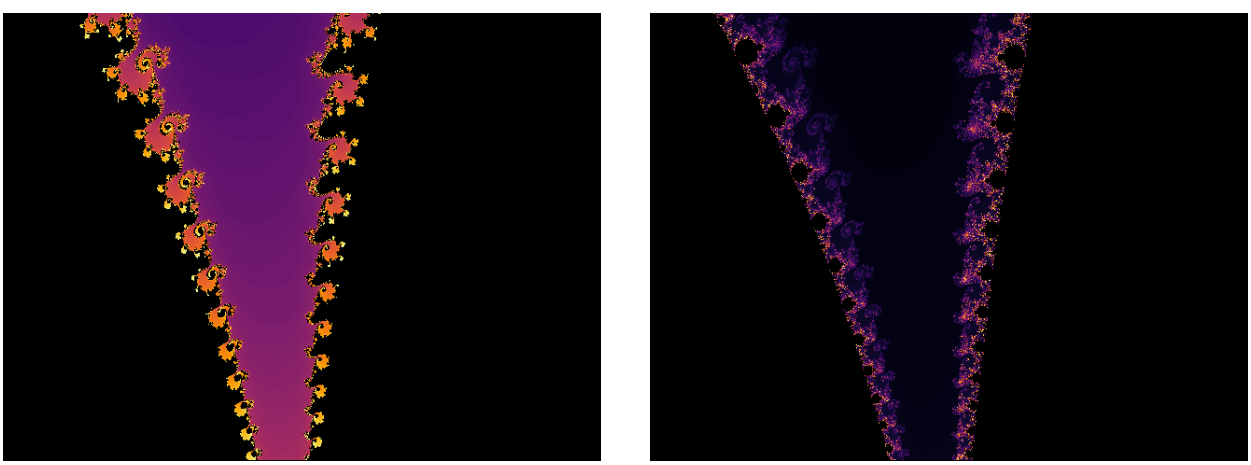
3.4.2 Integer bit width

In the Mandelbrot calculation, the magnitude of the complex number is checked with each iteration to see whether or not it has 'escaped'. If the magnitude $|z_n| \geq 2$, the complex number is known to diverge to infinity. In hardware, it would be simpler to check the magnitude squared so the escape condition would be $|z_n|^2 \geq 4$. Therefore, at least 3 bits are required to store the integer part of the number. To provide margin for overflow and to properly represent points outside the Mandelbrot set such as when visualizing the full set zoomed out, 4 bits were chosen for the integer part. This can represent the range of numbers from $[-8, +8)$.

3.4.3 Image quality

The quality of the image generated is influenced by both the maximum number of iterations and the precision of the fixed-point representation. Points near the boundary of the Mandelbrot set may take many iterations to exceed the escape threshold. If the `maximum_iterations` value is set too low, these points are incorrectly classified as part of the set, reducing the image's accuracy. Additionally, the number of fractional bits determines how precisely pixel coordinates are mapped to complex numbers. At higher zoom levels, lower-precision representations cannot distinguish between nearby values, leading to loss of detail.

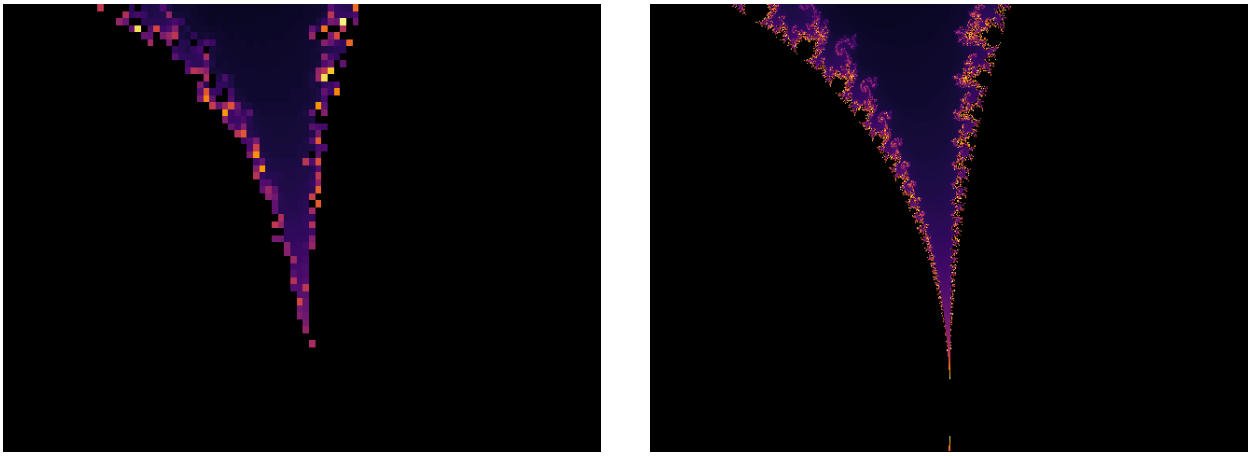
The figures below illustrate the effects of both insufficient iteration depth and limited numerical precision; Fig. 9 compares the images generated in simulation for 16 fractional bits with 100 and 1000 maximum iterations at a zoom level of $\times 32$, and Fig. 10 compares the images at a zoom of $\times 8$ for 8-bit and 16-bit fractional bits.



(a) 16-bit precision, 100 max iterations

(b) 16-bit precision, 1000 max iterations

Figure 9: Lower iterations fail to capture details of the fractal



(a) 8-bit fractional bits: limited detail

(b) 16-bit fractional bits: improved detail

Figure 10: Higher fractional precision enables clearer detail at high zoom levels

3.4.4 Word length

The DSP slices in the PYNQ-Z1 FPGA are 18x25 bit [8], hence any multiplication that fits in this bit width may be computed in one clock cycle. As the Mandelbrot equation involves squaring numbers, the maximum word length if one DSP slice is utilised is 18 bits to implement 18x18 multiplication. Higher word lengths would require more DSP slices connected in sequence, increasing the combinational delay which may cause timing violations.

However, higher word length multiplication can be achieved by computing 16x16 bit partial products separately and then combining them to assemble the full product. For example, for 32x32 bit multiplication $C = A \cdot B$:

$$A = A_{high} : A_{low} = A_{high} \cdot 2^{16} + A_{low}, \quad B = B_{high} \cdot 2^{16} + B_{low}$$

The partial products can be calculated as:

$$P_0 = A_{low} \cdot B_{low}, \;\; P_1 = A_{low} \cdot B_{high}, \;\; P_2 = A_{high} \cdot B_{low}, \;\; P_3 = A_{high} \cdot B_{high}$$

These can be combined to calculate the full product:

$$C = P_0 + 2^{16} \cdot (P_1 + P_2) + 2^{32} \cdot P_3$$

As each partial product is independent, they may be computed in parallel in one clock cycle. 16x16 bit multiplication was chosen even though 18x18 would also be supported, as 16 is a power of 2 making it well-suited for decomposing common word lengths such as 32 or 64 bits.

For the complex number multiplication associated with depth calculation, careful consideration was given to DSP slice utilization. If direct 32x32-bit multiplication was chosen in the `depth_calculator.sv` module with an additional wait stage to allow the products to settle, it would use 4 DSP slices as the synthesizer naively splits it into four 16x16-bit multiplications. The operations $\Re(z)^2$, $\Im(z)^2$ and $2\Re(z)\Im(z)$ would therefore require a total of 12 DSP slices (doubling the cross product $\Re(z)\Im(z)$ can be

achieved by a simple left shift).

However, a simplification may be made for $\Re(z)^2$ and $\Im(z)^2$ as two of the intermediate partial products would be the same; $A_{high} \cdot A_{low} = A_{low} \cdot A_{high}$. Therefore, if that partial product is calculated once and shifted left (to double it), the number of 16x16 bit multiplications needed for the squaring operations reduces to 3. Hence, the number of DSP slices required in `depth_calculator.sv` is reduced to 10. Therefore, directly calculating the partial products in the suggested manner would be more efficient in terms of DSP slice utilisation. This allows for more engines to be instantiated in parallel, increasing the overall throughput.

A word length of 32 bits could be used with 4 integer bits and 28 fractional bits. This represents a resolution of $2^{-28} \approx 3.72 \times 10^{-9}$, which supports a maximum zoom of $\frac{3}{640 \times 2^{-28}} \simeq 1.2 \times 10^6$ assuming a screen resolution of 640x480. A word length of 64 bits would represent a resolution of 2^{-60}, which supports a maximum zoom of $\frac{3}{640 \times 2^{-60}} \simeq 5.4 \times 10^{15}$.

A 32×32-bit multiplication can be implemented using four 16×16-bit partial products, whereas a 64×64-bit multiplication requires sixteen 16×16-bit partials. As a result, a 64-bit fixed-point engine would either require approximately four times the number of DSP slices compared to a 32-bit engine, or incur a greater latency per iteration if the same hardware resources are reused across cycles. In both scenarios, the overall throughput is reduced, either due to a lower number of parallel engines that can be instantiated within the hardware constraints, or due to the increased number of clock cycles required to complete each iteration.

For the intended purpose of a general Mandelbrot set explorer, 32 bit words provide sufficient accuracy.

3.5 Multi-engine Implementation

One of the most effective ways to increase algorithmic throughput is to parallelise the calculation of pixel depth values, taking advantage of the *embarrassingly parallel* nature of the Mandelbrot set.

Initially, a large RAM block was instantiated as a line buffer, with each pixel assigned a dedicated memory location. The x coordinate served as the address, and the corresponding depth value, not the full RGB data, was written to and read from RAM to minimise memory usage. However, even with this optimisation, BRAM limitations on the FPGA prevented the design from scaling to higher resolutions. This constraint necessitated the development of an alternative parallelised implementation.

The existing single-engine module was reused, each engine with its own `pixel_to_complex.sv` module and FIFO. Each engine outputs the calculated depth value, concatenated with the pixel's x and y value into the FIFO. The output of each engine's FIFO is compared with the current x and y values from the pixel generator. When a match is found, the corresponding FIFO is read to retrieve the depth value. The number of engines is easily configurable via a single parameter, and all engines are instantiated within a dedicated `engine_top.sv` module to ensure modularity, scalability, and a clean design hierarchy.

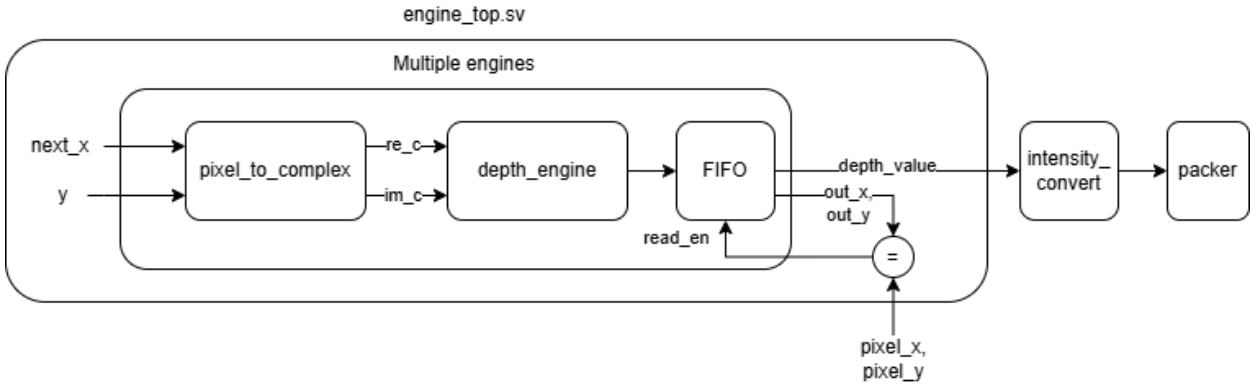


Figure 11: Block diagram of Multi-engine architecture

3.5.1 Design Considerations

Parallelising the workflow introduces several design challenges:

- How should workload be allocated across engines?
- How are control signals coordinated among multiple engines?
- How can pixels be sent downstream in the correct image order?
- How can limited FPGA resources be managed efficiently?

3.5.2 Workload Allocation

First-Come, First-Served (FCFS) was selected as the workload allocation strategy. This approach is well-suited to the Mandelbrot algorithm, as the number of iterations per pixel varies widely; Some pixels escape after only a few cycles, while others require close to the maximum iteration limit. FCFS ensures that engines are not stalled by uneven workloads; each engine takes on the next available pixel as soon as it becomes idle, maximising utilisation. The assigned coordinates are also independent of the pixel required by the `pixel_generator.v` module, ensuring that pixels are always ready to be read.

The main drawback of FCFS is that output pixels are completed in a non-deterministic order. Since engines operate independently, the order in which results become available does not match the raster order of the image. To preserve the correct output sequence, a FIFO buffer was introduced to each engine. It temporarily stores completed pixel values before they are emitted in the correct order.

3.5.3 FIFO

The backpressure signal `ready` from the stream receiver is asserted non-deterministically and data can only be read if required pixel is available and ready is asserted. This introduces a potential issue: if the rate at which data enters each FIFO exceeds the rate at which it is read, the FIFOs may eventually overflow. This phenomenon is captured by the concept of *traffic intensity* ρ and is defined as such:

$$\rho = \frac{\text{arrival rate}}{\text{service rate}}$$

When $\rho > 1$, data enters the FIFO faster than it can be read out, causing the queue to grow indefinitely and eventually overflow. Since service-rate is random, it is not guaranteed that the arrival-rate of pixels to

each FIFO will remain below the service-rate. To prevent data loss, each FIFO implements a back-pressure mechanism: when the FIFO is full, it asserts a signal to its corresponding engine to stop accepting new jobs. This flow control ensures safe operation by pausing computation until space becomes available in the FIFO.

The number of iterations required for each pixel in the Mandelbrot set varies widely, meaning that engine completion times are heavily staggered. As a result, the effective traffic intensity is typically well below 1. This allows a FIFO of depth 16 to provide sufficient buffering capacity to absorb bursts of inputs without excessive stalling. The design balances theoretical peak throughput with real-world variability, ensuring reliable operation without incurring excessive hardware cost.

3.5.4 Pixel Ordering

To order the pixels in the correct raster order, the x and y values from the `pixel_generator.v` are input into `engine_top.sv`. These are compared with the output x and y values from each FIFO and when a match is found, the data is read from the FIFO and then waits for the ready signal to emit the pixel.

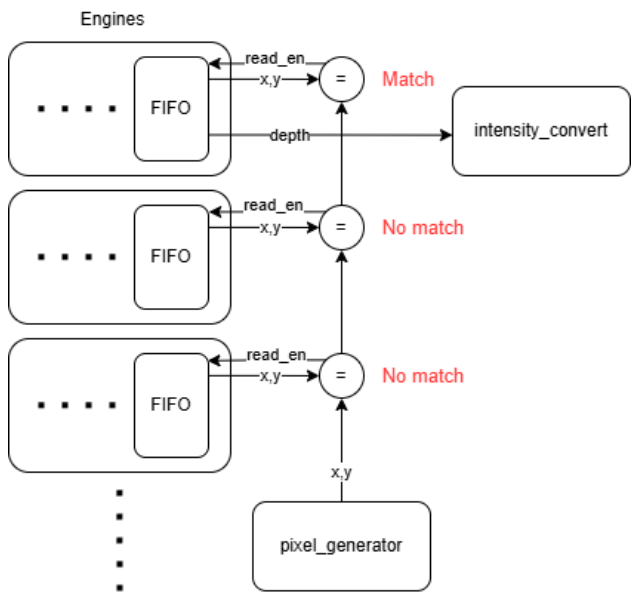


Figure 12: Diagram of pixel ordering by matching the required coordinate and fifo coordinate

3.5.5 Number of engines

The number of engines that could be instantiated heavily depended on timing errors and resource availability on the FPGA. A very negative slack could lead to the compiled bitstream not functioning, and exhausting FPGA resources led to errors in device compilation.

Increasing the number of engines made the worst negative slack (WNS) and total negative slack (TNS) to become more negative. This was because paths from shared registers to each individual depth engine in `engine_top.sv` had to be routed to engines that were physically a greater distance away on the FPGA fabric from the registers.

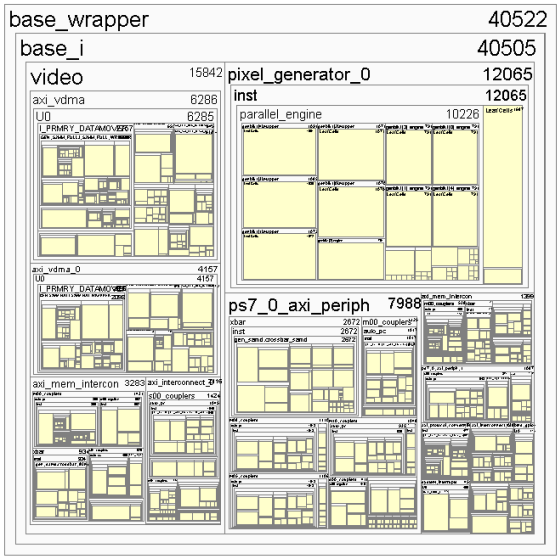


Figure 13: Visualization of resource distribution on the FPGA for 5 engines

To maximise available FPGA resources, many of the IP blocks that came with the original design, such as those providing PMOD connectivity to the board, were removed. By removing these we greatly reduced the number of slices and DSPs used by IP other than our pixel generator. This led to our main constraint on how many engines we could instantiate being the 220 available DSPs on the board. A single engine with a mapper utilises 14 DSP slices. Therefore, for 220 DSP slices on the PYNQ-Z1 board, a maximum of 15 engines can be instantiated.

3.5.6 Implementation & improvements

When the multi-engine solution was implemented with the optimized `pixel_to_complex.sv` module described in Section 3.2.5, the output image did not render as expected. This was because the optimized `pixel_to_complex.sv` expects the pixels (x, y) to be input in sequential order; This is an assumption violated by the parallel architecture, where engines receive pixels in arbitrary order. As such, it was necessary to use multiplication as in the initial `pixel_to_complex.sv` iteration.

However, the direct multiplication is known to cause timing violations (see Section 3.2.5) so a similar approach as described in Section 3.3.4 is used where the multiplication is split into partial products, which are assembled into the full product one clock cycle later. This incurs a one clock period latency per pixel, but it does not affect the depth engines' functionality, as they can process the complex number a clock cycle later without issue. The `pixel_to_complex.sv` module now uses 4 DSP slices to implement these multiplications.

For Zynq-7000 FPGAs, Block RAM (BRAM) can be used to implement FIFOs, with each FIFO typically mapped to a single BRAM resource [9]. This approach was considered to help conserve LUTs and flip-flops, making more logic resources available for computation. However, this optimisation was ultimately deemed unnecessary, as LUT usage was no longer a limiting factor in the overall design and forcing the use of BRAM could lead to a greater negative slack for paths accessing the FIFOs.

Additionally, it was also necessary to make a slight modification in the `pixel_generator.v` module to

initialize the regfile with values at the start. The images did not render without the initial values being set; this was likely due to some internal logic within the Mandelbrot computation being broken when `MAX_ITER` is set to 0, preventing the appropriate signals from advancing and not allowing `valid_int` to go high. This means no frame was generated as the Jupyter notebook seemed to load indefinitely without producing any output.

```
    initial begin
    // Default MAX_ITER
    regfile[0] = 32'd512;
    // MAX_ITER_LOG (log2(MAX_ITER))
    regfile[1] = 32'd9;
    // Default ZOOM
    regfile[2] = 32'd1; // Example: ZOOM = 1.0
    // Default REAL_CENTER
    regfile[3] = -(32'd3 * (32'd1 << (FRAC-2)));
    // Default IMAG_CENTER
    regfile[4] = 32'd0;
end
```

3.6 Julia Set

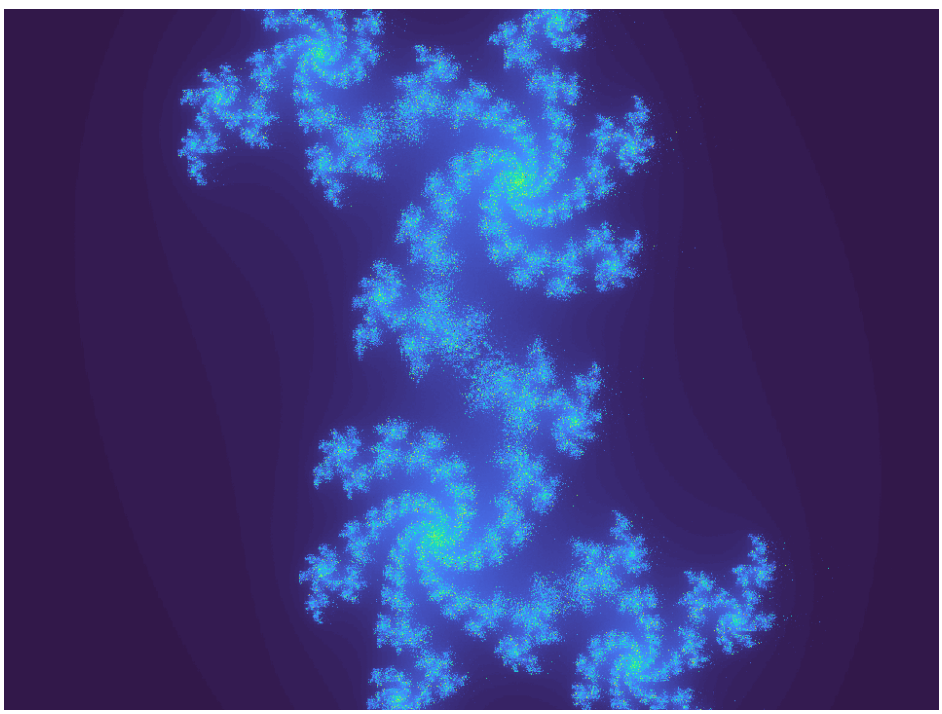


Figure 14: Julia Set produced from our multi-engine implementation

Due to the shared recurrence relation between the Julia and Mandelbrot set, with a few simple modifications the Julia set can be displayed. The difference between the two is that the Mandelbrot starts with $\Re(z_n) = \Im(z_n) = 0$ and varies the complex c while each Julia set is instead formed by some fixed complex c while $\Re(z_n)$ and $\Im(z_n)$ are changed to form the image.

$$z_{n+1} = z_n^2 + c, \quad z_0 \in \mathbb{C},\ c \in \mathbb{C}$$

These involved adjusting the Q format to allow a wider range of values to be displayed as well as adjusting the iteration logic in depth calculator to properly compute the recurrence relation for the Julia set. Since we could already generate a raster pattern of complex numbers all that was required was to change the input from 0 to those generated by the pixel to complex module and have c a variable input allowing us to view the range of Julia sets.

4 Webpage Implementation

4.1 System Overview and Design Choices

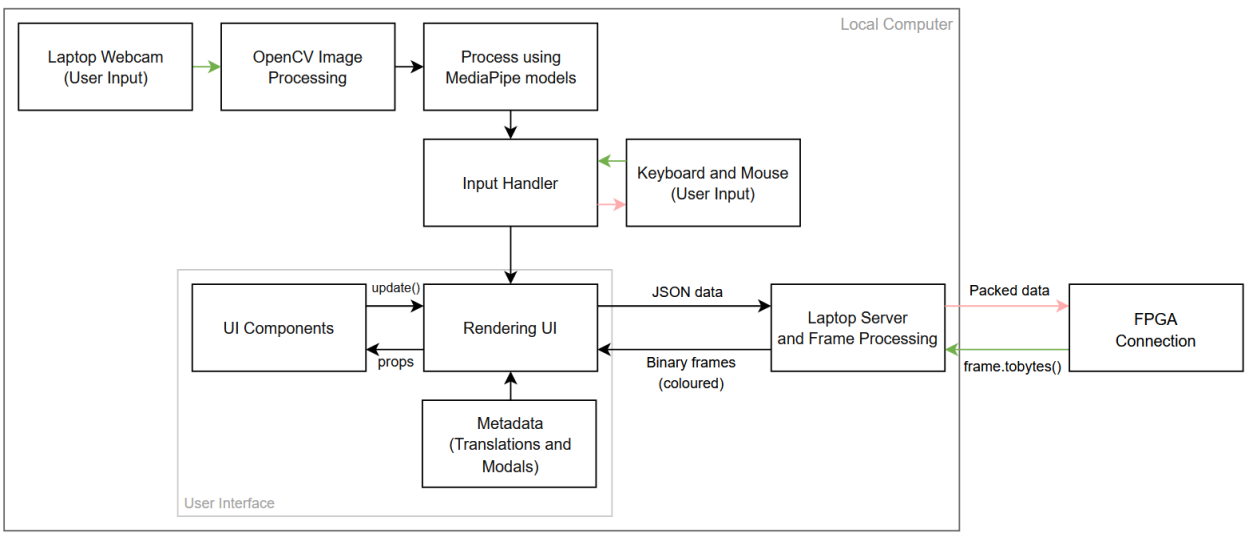


Figure 15: Software System Diagram

The key aim of the user interface (UI) is to provide a means of enhancing the visualisation as an educational tool. For a pleasant user experience, ensuring the UI is responsive and easy to navigate are key focuses upon the design process of the UI. SolidJS and TailwindCSS [7] were used to develop the UI, allowing for fine-grained reactivity and efficient compilation - due to the absence of diffing in Virtual DOMs, unlike other JavaScript frameworks. Only reactive components are updated, through signal tracking and compile-time optimisations - resulting in improved performance: making it a suitable choice of framework.

The application is deployed locally on a laptop, with user interaction facilitated through standard peripherals including the keyboard, mouse, and camera. These input devices allow the user to control and manipulate the view — such as zooming, panning, and modifying parameters — which are then used to generate corresponding frames on the connected hardware.

4.2 User Interactivity

The user can interact with the Mandelbrot set displayed using a mouse / track pad and keyboard on the webpage. Hovering over a visible point will display the complex number represented, and to provide a smooth and intuitive user experience, scrolling inward will allow the user to zoom in on the set centred about the complex value highlighted by the cursor at that moment. Scrolling outward will also zoom out

of the set, with the centre also being the hovered over point. To provide a maximally educational experience of the complexities of the Mandelbrot set, key areas of the set are mapped to buttons, meaning the user can easily traverse to areas of interest which include the spiral arms, seahorse valley, circular bulb and main cardioid at specific points and zooms.

Initially, FPGA-generated frames were incorporated as a small canvas within the entire webpage. However, frames became the background through webpage redesign; enabling scalability when testing different canvas dimensions and full utilization of the visual space. In the bottom left corner, $re(c)$, $im(c)$ and the zoom level are updated in response to user input. A collapsible dropdown menu can be toggled with the top button, where buttons represent: colouring schemes, maximum iterations, ROI and general settings. When exploring certain regions of interest within the Mandelbrot set, a button is triggered in the bottom right corner; clicking this will expand a modal that provides information regarding these regions - enhancing the educational value of the mathematical visualization.

4.3 Webpage

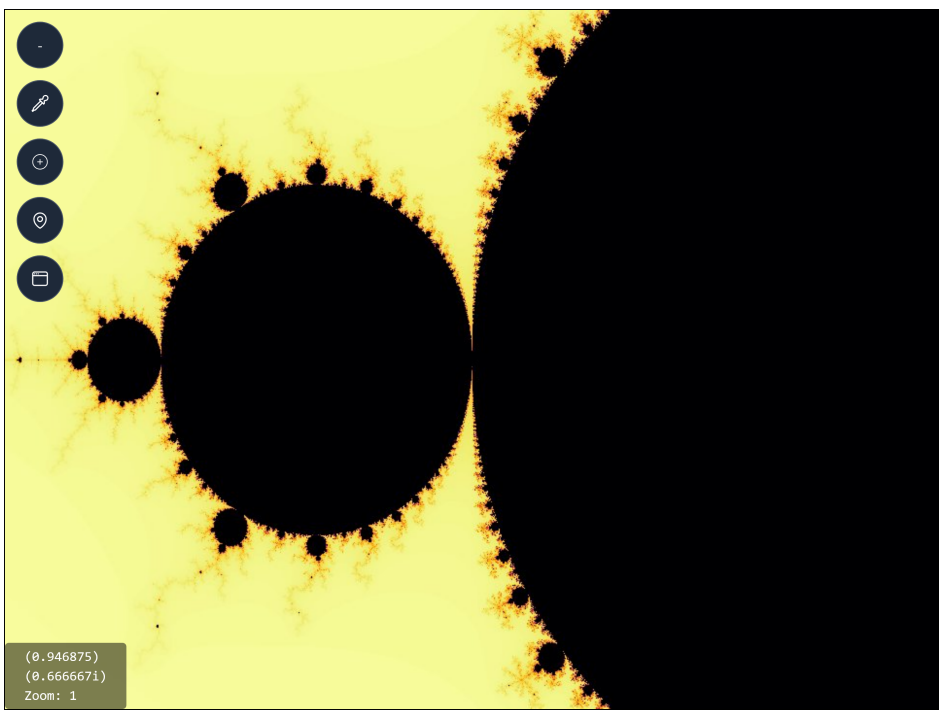


Figure 16: Webpage with frame loaded

SolidJS was selected as the primary framework for incorporating components and web elements to form the UI contruction. As seen in the listing above, the background of the layout is the frame generated in hardware - with an interactive dropdown menu positioned along the left-hand side of the webpage, providing users with comprehensive control over application functionality. These buttons facilitate user inputs and accessibility features, through hover and click handlers. In the bottom left corner, a dynamic information panel reads the center values and zoom corresponding to the mouse cursor on the screen. The webpage incorporates a comprehensive modal system that presents contextual information and usage instructions. Interaction with buttons toggles the modals, providing readable content that reinforces key

information related to the Mandelbrot set.

Utilising SolidJS's key functionalities, functional elements (components) are used, so that a repeated element can be reused with varied content - improving code maintainability and reducing implementation redundancy across the webpage. Standardised components were written for modals (pop-up boxes) and buttons that have a set frame, but support custom content through use of props.children. An example of this is through the use of buttons in the cascading menu structures, implemented as separate components to accommodate varying animations and content requirements. Integration of animation into the menu system enhances the user experience through smooth and refreshing transitions. These animations serve as visual feedback to user interaction, indicating state and signal changes, whilst also serving an aesthetic purpose.

The styling architecture utilises Tailwind CSS rather than traditional inline CSS approaches, providing a class-based styling system that ensures visual consistency across all interface components. Tailwind's compilation process optimises the final application by removing unused styles, preventing DOM bloating and improving overall webpage performance at runtime. This optimisation directly contributes to faster loading times and improved user experience.

4.4 Parameterising Signals

4.4.1 Using peripheral I/O

For the frame generated in the Mandelbrot program to change through parameterisation, usage of keyboard and mouse is required to interact with the UI components. Hover and click handlers are used and initialised using the `createSignal` method within the top `App.jsx` file, where all the UI elements are put together. These are defined within the function as core states:

```
const pos = createMousePosition(window);
const [mouseWheelDelta, setMouseWheelDelta] = createSignal(1);

const [centerX, setCenterX] = createSignal(-0.5);
const [centerY, setCenterY] = createSignal(0.0);
```

Listing 6: Initialising signals for I/O related inputs

where `createMousePosition` tracks the mouse coordinates relative to the window position and updates the Mandelbrot frame parameters via the setter function `setCenterX` and its coplanar equivalent. `mouseWheelDelta` [3] captures the zoom level, which is modified in response to scroll events. This peripheral interfacing allows for reactive updates to the Mandelbrot visualisation in realtime.

4.4.2 Regions of Interest

To provide an educational experience on the webpage, several key regions of interest of the Mandelbrot set are mapped, including the Main Cardioid, Circular Bulb, Spiral Arms, Seahorse Valley, and Elephant Valley. Each of these areas have an icon which takes you to the region of interest and a detailed mathematical description of how it is formed from the Mandelbrot equation. To ensure a smooth user experience, the frames of each of these are precomputed and cached for immediate access.

One of the programmed cascade components stores mathematical features of distinct regions of the

Mandelbrot set, including values for the center coordinates and the zoom exponent. An example region can be seen in the listing below:

```
const regions = {
    seahorse: { centerX: -0.74, centerY: 0.13, zoom: 7 },
```

Listing 7: storing values for the Seahorse Valley

The region selection mechanism implements direct transmission of parameters to the parent state in `App.jsx` through the use of the SolidJS `onJump` function, which updates the relevant parameters. This triggers the same reactive update chain that handles manual coordinate and zoom inputs.

```
const regionChange = (regionKey) => {
    const location = regions[regionKey];
    props.onJump(location);
};
```

Listing 8: Updating location values

4.4.3 Signal constraints

As mentioned above, certain parameters require specific input formats, due to differing bit widths and fixed-point requirements. To ensure compliance with the hardware, and prevent overflow in combinational logic, inbuilt math functions are used to bound some of the signals, with values that can be altered by user input.

The maximum number of iterations has a range of values that spans 0, 10, so using `Math.max` was required to set the iteration counter.

The zoom exponent must be transmitted as an integer value, hence using `Math.floor` is required to truncate the fractional part of the mouse scroll signal. Only the exponent is transmitted, to reduce channel load between the local host and FPGA, at which point the zoom can be calculated.

4.4.4 State architecture and props

The state management architecture for inter-component interfacing was employed using SolidJS props. This is a reactive system that guarantees data consistency across the entire DOM component structure. Rather than directly passing signal references, props are used for reliable state propagation; parents are re-rendered when states are used or changed, which triggers component updates in the children/related components. This methodology guarantees reactivity and eliminates signal deferencing, that can occur as a result of direct signal passing.

4.5 Websockets and Asynchronous functionality

Using SolidJS onMount hooks ensure event listeners and websocket connections are configured correctly, providing correct system initialisation and signal management. In the UI, onMount is used for mouse wheel event handling and dual websocket connection initialisation.
The former uses `onMount` to implement mouse wheel event handling through control of the input `event`, in which the zoom is written as a function of its previous value. Within this hook, the zoom control utilises bounding functions to constrain zoom values, preventing overflow conditions and updating values in a form suitable for propagation to the FPGA. The body of the hook can be seen below:

```
const handleWheel = (event) => {
    const newValue = Math.floor(mouseWheelDelta() + (-event.deltaY) * 0.01);
    setMouseWheelDelta(Math.max(0, Math.min(newValue, 32)));
};
```

Listing 9: Updating location values

Alternatively, a `onMount` hook manages dual websocket connections - one of which transmits parameters to the FPGA, and the other receiving frames for use on the UI. This implementation instantiates two distinct target endpoints, the laptop processing server `localhost:8001` and the FPGA server `192.168.137.255/24:8080` for realtime parameter updates.

The application uses asynchronous functions to manage concurrent communication and non-blocking I/O operations within websockets in the system hierarchy. Using `await` and `async` [6] is essential for handling real-time data transmission between the FPGA and relevant components within the local host (laptop). Most websocket code within the system uses `try/catch` blocks that handle connection failures and communication timeouts. This is so that the independent programs are not required to be restarted during runtime - due to these mitigating methods.
Using asynchronous functions allows interactions in the UI to be responsive, through frame generation - as parameter changes trigger websocket transmission immediately without buffering of any fractal calculation. This allows the returned frame to update in real-time with minimum input lag.

4.6 Communication with FPGA

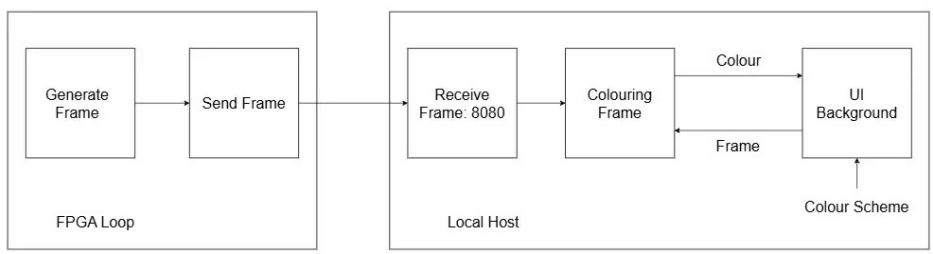


Figure 17: FPGA to local host full connection

```
def apply_colormap(gray_image, cmap_name):
    try:
        cmap = cm.colormaps[cmap_name]
    except AttributeError:
        cmap = cm.get_cmap(cmap_name)

    normed = gray_image / 255.0
    colored = cmap(normed)[..., :3]
    return (colored * 255).astype(np.uint8)
```

Listing 10: Function filtering grayscale image

These defined functions are integrated through a unified function called upon reception of the frame's binary data.

```
async def process(websocket):
    async for message in websocket:
```

```
        try:
            recv = np.frombuffer(message, dtype=np.uint8).reshape((720,960,3))
            new_frame = colormapped_image(recv, cmap_name='inferno')
            await send_to_ui(new_frame)
```

Listing 11: Decoding binary message for colouring

The `process` function implements a continuous asynchronous processing loop that handles incoming binary data from the FPGA hardware. To maintain a persistent connection, `async for` [5] is used to monitor incoming data as it arrives. Using this iteration structure enables non-blocking frame reception; frames can still be processed without blocking the event loop or interfering with other websockets' connections in the application.

Upon recieving the websocket message (using `np.frombuffer(message)`, there is binary data reconstruction to obtain the data as a NumPy array, its original format.

The processed frame immediately enters the asynchronous transmission pipeline for delivery to the UI. Using an `await` method ensures realtime execution - so that frames can be viewed on the webpage with minimal latency.

4.6.1 Use of Binary encoding

During the implementation of frame transfer from the FPGA to the user interface (UI), two data encoding methods were evaluated: raw binary transmission and Base64 encoding. After careful consideration, binary transmission over WebSockets was selected as the preferred method due to its greater resource efficiency and higher transmission speed.

Unlike Base64 encoding, which introduces a roughly 33% increase in payload size due to character-based representation, raw binary transmission retains the original size of the data. This eliminates unnecessary overhead and ensures that bandwidth is used optimally. Additionally, binary transmission avoids the computational cost of encoding and decoding steps, thereby minimising latency and improving responsiveness in the UI. This reduction in latency is particularly advantageous for applications involving real-time frame updates, where delays can result in perceptible lag or static displays. By using binary data directly, the system can transmit frames more quickly and reliably, enhancing the overall user experience.

The frames are read from the FPGA as NumPy arrays, which are encoded using Python's native `.tobytes()` function and decoded on the receiving end with `numpy.frombuffer()`. This method leverages widely supported, efficient functions for seamless binary data handling between the FPGA and UI.

4.6.2 Communication within the localhost

The processed frames are used as the background for the UI, implementing Blob URL for handling transmitted frames from the local processing server. The system recieves binary JPEG data as Blob objects, converting them to temporary object URLs using the `URL.createObjectURL()` static method, subsequently assigned to the background image of the UI container, allowing seamless visual updates.

Through use of Blob URLs, there is minimal computational overhead in encoding and decoding, whilst maintaining close-to-original quality. For efficient memory management, unused or redundant URLs can be released using `URL.revokeObjectURL()`, preventing memory leaks and ensuring optimal frame-loading performance at runtime. This API method is retained after a timeout period of five seconds; explicitly revoking the URL (before browser intervention).

4.6.3 Communication from UI to FPGA

To facilitate the control of parameters for frame generation, the signals `centerX()`, `centerY()`, `mouseWheelDelta` and `iterCounter()` are monitored and serialised for transmission in JSON format. The reactive primitive `createEffect` monitors changes in these state variables, triggering automatic execution whenever any dependent signals are updated. This mitigates the role of polling or explicit event handling - parameter updates are immediately modified and propagated to the FPGA. Leveraging this primitive simplified the code, whilst reducing latency between manual user input and hardware configuration, for immediate generation of frames.

To ensure that redundant or erroneous data is not transmitted, the condition that wraps `ws.send()` ensures that the websocket is open and that the FPGA is in a ready (to recieve) state. This ensures robust communication and synchronisation between the UI and FPGA.

```
if (ws?.readyState === WebSocket.OPEN) {
    ws.send(JSON.stringify({
    re_c: centerX(),
    im_c: centerY(),
    zoom: mouseWheelDelta(),
    max_iter: counter(),
    }));
```

Listing 12: Sending parameters in JSON format

The FPGA listens for JSON messages at the websocket endpoint, extracting the values to load into the hardware registers (as input parameters for frame generation). From the UI, the values are transmitted as floats; before loading values into registers, there must be conversion into a compatible format for the hardware. Most parameters can be wrapped and converted using `int()`, whilst `im_c` and `re_c` must be converted into Q4.28 fixed-point format to reduce resource usage and increase calculation speed during fractal computation. This conversion is handled by the function seen in the listing below, ensuring compatibility with the hardware.

```
def float_to_q4_28(fval):
qval = int(round(fval * (1 << 28)))
if qval < 0:
    qval = (1 << 32) + qval
return qval
```

Listing 13: Making float values Q4.28 compatible

4.7 Gesture recognition

To enable interactivity with the user interface using hand gestures, OpenCV and MediaPipe were used in Python to allow zooming and panning of the Mandelbrot set via pinching and pointing. OpenCV [4] was

used for video capture, image processing and display as it is fast and reliable, and MediaPipe [2] was used to detect real time hand gestures due to its easy integration, pre-trained models and high accuracy, allowing for as intuitive interface as possible.

The zooming is done by recognising a pinching gesture, which zooms by measuring the Euclidean ditance between the wrist and index finger, and normalising this to ensure it is accurate for any distance.

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

This is translated to a zoom value between 0.5 and 1.5 by scaling the distance by a factor of 6.

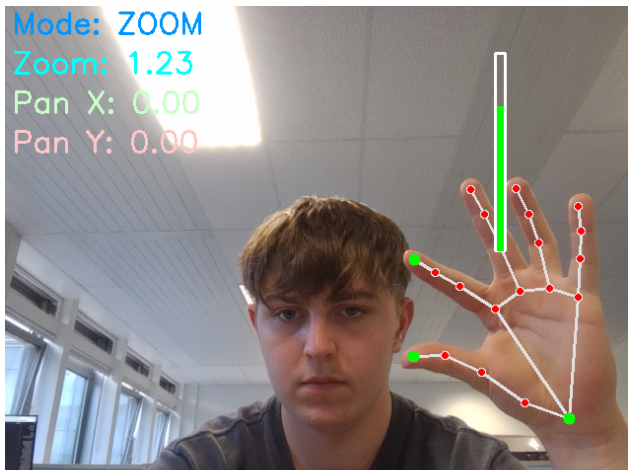


Figure 18: Use of MediaPipe to Zoom

If the zoom value is 1 or above, and is held for 0.8 seconds, it will update the zoom by a factor of 2. If it is below o.8 and held for o.8 seconds, the zoom decreases by a factor of 2, providing an easy to use and intuitive zoom system using hand gestures.

4.8 Colouring

The frame processing program implements a three-stage colour mapping pipeline that transforms raw grayscale intensity values into fractal data from the FPGA hardware into coloured frames for web-based display, based on the selected scheme. This processing methodology leverages scientific visualization techniques through matplotlib's `colormap` system to enhance the mathematical structures present in Mandelbrot set iterations.

The first stage uses channel extraction to obtain a single channel, which forms the base intensity map, as all channels per frame share the same value.

```
1 def rgb_to_grayscale(rgb_image):
2     return rgb_image[..., 0]
```

Listing 14: Extracting a single channel

Following this, the colormap system applies uniform colour gradients to the intensity values. To prevent issues with version compatibility, a try-exception structure was used to handle different releases.

4.9 EDI

Ensuring the UI was accessible to wide range of users was a primary objective, upon which was achieved through implementing features such as customisable colour schemes, multilingual support and a text-to-speech reader. To improve visual clarity, various colour schemes could be chosen - facilitated through coloured buttons that filter the raster frame when hovered over. In order to support multilingual compatibilty, the program supports English, Spanish, Mandarin and Arabic, enabling users to read modal content in their preferred language. All text metadata, such as translations and titles, are stored locally in the `translations.js` file. This approach avoids reliance on live translation services, ensuring consistent output and reducing latency during page loads. Text-to-speech functionality was integrated to enhance further accessibility. Within modals containing textual information, buttons are embedded that allow users to trigger audio playback. Additionally, speech cadence and language compatibility was refined through use of API calls tailored to the supported languages.

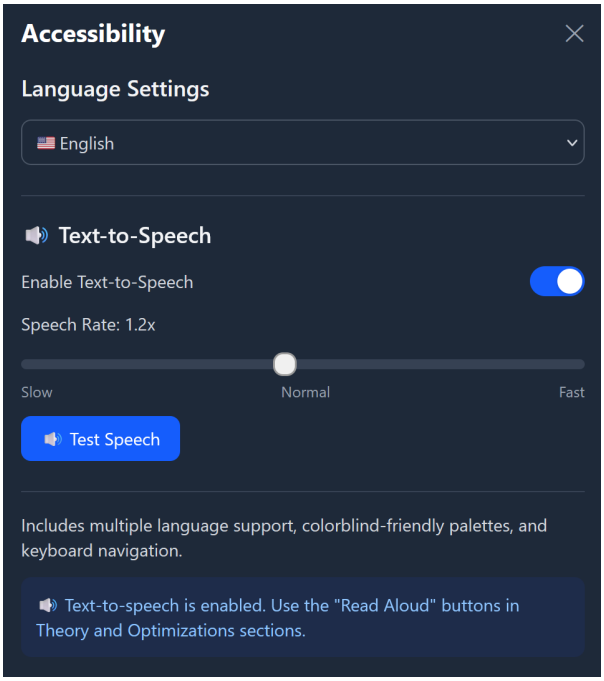


Figure 19: TTS and language settings

Figure 20: Optimisations in Arabic

5 Software implementation

5.1 C++ implementation

```
void renderFractalPart(int startRow, int endRow) {
  for (int y = startRow; y < endRow; ++y) {
    for (int x = 0; x < WIDTH; ++x) {
      double realPart = REAL_MIN + (REAL_MAX - REAL_MIN) * x / WIDTH;
      double imagPart = IMAG_MIN + (IMAG_MAX - IMAG_MIN) * y / HEIGHT;
      std::complex<double> c(realPart, imagPart);
      std::complex<double> z = 0;
      int iter = 0;

      while (std::abs(z) <= 2 && iter < MAX_ITER) {
```

```
        z = z * z + c;
        iter++;
      }

     image[y * WIDTH + x] = iter;
    }
  }
}
```

Listing 15: Mandelbrot generating function

The listing above shows the main function `renderFractalPart` that iterates through each pixel in the frame, using a nested for-loop structure. Using `std :: complex < double >`, both real and imaginary parts can be isolated and defined individually, and simplifies the implementation as there is no need to process either separately - enabling computation through the function that defines the Mandelbrot set in line 11.

```
void renderFractal() {
  std::vector<std::thread> threads;
  int rowsPerThread = HEIGHT / NUM_THREADS;

  for (int i = 0; i < NUM_THREADS; ++i) {
    int startRow = i * rowsPerThread;
    int endRow = (i == NUM_THREADS - 1) ? HEIGHT : (i + 1) * rowsPerThread;
    threads.push_back(std::thread(&MandelbrotSet::renderFractalPart, this, startRow, endRow
    ));
    }

  for (auto& thread : threads) thread.join();
}
```

Listing 16: Multithreading Mandelbrot function

The `renderFractal` function parallelizes fractal rendering by dividing the image into horizontal strips, each processed by a separate thread. It initializes a vector of threads `(std::vector<std::thread>)` and calculates the number of rows per thread by dividing the total height by the number of threads. For each thread, it determines the start and end rows, ensuring the last thread covers any remaining rows by clamping`endRow` to HEIGHT. Each thread executes `renderFractalPart`, a member function of the `MandelbrotSet` class, which computes the fractal for its assigned row range. After launching all threads, the function waits for their completion via `thread.join()`.

5.2 Cython implementation

```
def mandelbrot_cy(double complex c, int max_iter):
    cdef double complex z = c
    cdef int n
    cdef double abs_z

    for n in range(max_iter):
        abs_z = sqrt(z.real * z.real + z.imag * z.imag)
        if abs_z > 2.0:
            return n
        z = z * z + c
    return max_iter
```

```
12
13 def mandelbrot_set_cy(double xmin, double xmax, double ymin, double ymax,
14                        int width, int height, int max_iter):
15     cdef cnp.ndarray[ITYPE_t, ndim=2] result = np.zeros((height, width), dtype=np.int64)
16     cdef double x_step = (xmax - xmin) / width
17     cdef double y_step = (ymax - ymin) / height
18     cdef int i, j
19     cdef double x, y
20     cdef double complex c
21
22     for i in range(height):
23         y = ymin + i * y_step
24         for j in range(width):
25             x = xmin + j * x_step
26             c = x + 1j * y
27             result[i, j] = mandelbrot_cy(c, max_iter)
28
29     return result
```

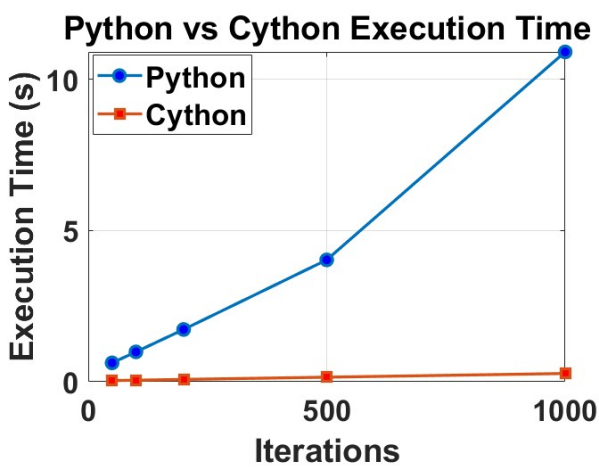
Listing 17: Cython Mandelbrot implementation

A single-threaded Cython implementation was also included for a software comparison. `mandelbrot_cy` checks each pixel individually to see after how many iterations it escapes. The `mandelbrot_set_cy` generates an image of the Mandelbrot set outputted as an array by looping over each pixel. Cython was used as opposed to python to avoid unnecessary overhead and increase speed.

6 Testing and Evaluation

6.1 Benchmarking

The software implementations of Python and Cython were bench marked for different numbers of iterations for a fixed center point and zoom. The Cython implementation had a significantly better performance than the Python; as number of iterations increases, the execution time for Python increases rapidly whereas the execution time of Cython grows much more slowly.

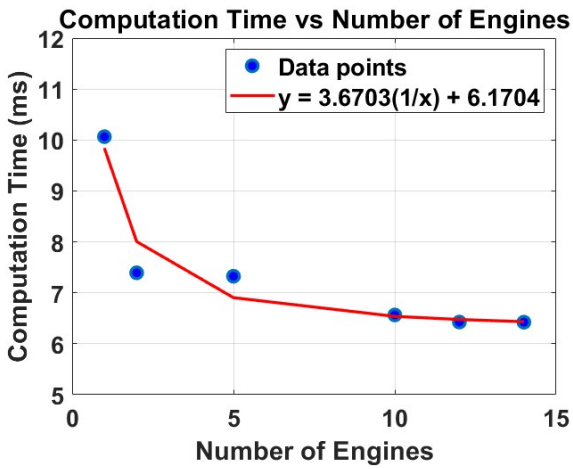


The hardware throughput will be analysed by investigating how much time each number of engines requires to generate the same frame. The following input parameters will be set:

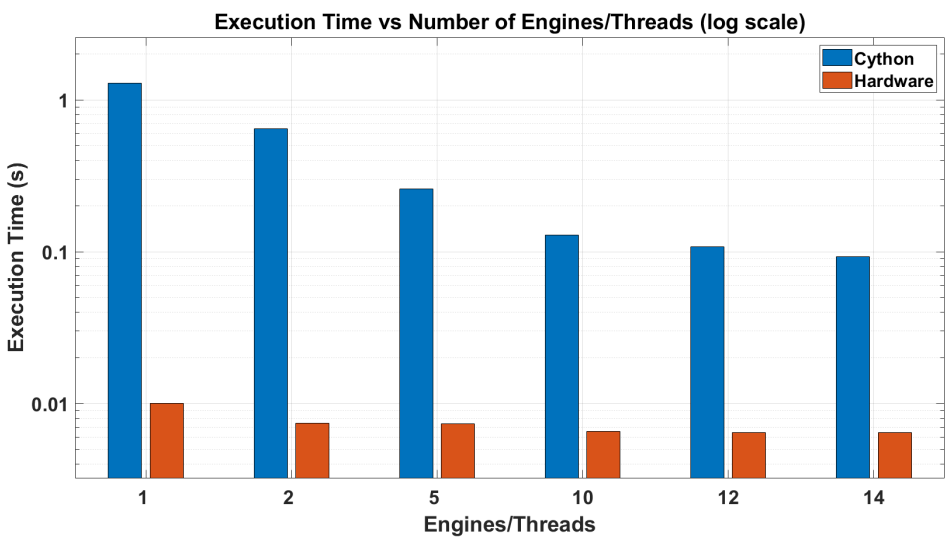
- `MAX_ITER = 256`
- `ZOOM = 0`
- `REAL_CENTER = -0.75`
- `IMAG_CENTER = 0.1`

The frame will be generated 100 times in each case, and the average time will be taken.

Owing to the embarrassingly parallel nature of the problem, a $\times$N speed-up is expected for N engines. A $\sim \frac{1}{N}$ relationship is observed with an offset of 6.1704ms. This suggests a significant sequential bottleneck in the system; this may be from from the `packer.v` module which can only accept one word per clock cycle. The packer outputs a maximum of one pixel per clock cycle, which gives a 7ns/pixel = 4.84ms/frame sequential delay. Other sequential operations e.g. VDMA setup can also contribute to this effect.



The plot shown below compares the execution time between the Cython and hardware implementations across different numbers of engines/threads. The results show a dramatic difference in performance as the hardware generates frames several orders of magnitude faster than the software.
As such, it was necessary to use a logarithmic scale on the y-axis as otherwise the hardware's execution times would appear almost flat compared with the software times. The figure also shows how the improvement in performance diminishes as parallelism increases, although hardware consistently performs better than software across all numbers of engines/threads.



6.2 Future improvements

6.2.1 Pipelining

The `depth_calculator.sv` module was divided into sequential stages to help meet timing; however, it was not practical to pipeline each engine. This is because each pixel's associated complex number requires an unpredictable number of iterations to reach the escape condition. As a result, it is impossible to determine in advance when it would be appropriate to accept new values into the pipeline, due to the iterative loop back within the state machine. However, in the future, the structure may be redesigned to allow for pipelining, which would increase throughput.

6.2.2 Pixel packer redesign

Using the current `packer.v` module provided introduced a bottleneck on the system throughput as it could only output a maximum of one 32-bit word per clock cycle.
For an N engine implementation, if multiple results arrive simultaneously, the `packer.v` module's output rate would limit the system. If this module were to be redesigned in the future to simultaneously output N pixels, the sequential delay could be reduced and the computation time per frame could be further minimised.

6.2.3 Parallel architecture redesign

Each engine currently includes its own dedicated `pixel_to_complex.sv` mapper, which generates the corresponding complex number for depth calculation. Since each mapper consumes 4 DSP slices, this design limits the number of engines that can be instantiated to $\left\lfloor \frac{220}{10+4} \right\rfloor = 15$, based on the total DSP slice availability.
If the parallel architecture were redesigned to use a single central mapper that supplies complex numbers to engines on demand, the maximum number of engines could increase to $\left\lfloor \frac{220-4}{10} \right\rfloor = 21$. However, benchmarking showed that performance gains diminished with additional engines, and the overall speed-up was limited. This change would only be worthwhile if other sequential bottlenecks in the system were resolved. Given these findings and time constraints, the optimisation was ultimately not pursued.

6.2.4 Julia Set Implementation

Currently it is possible to display the Julia set separately to our Mandelbrot set. However for a more compelling demonstration, after more development, it is intended to be able to display both simultaneously, showing the corresponding Julia set for a selected point on the Mandelbrot set. This would also allow demonstration of the property of connected Julia sets being formed from points within the Mandelbrot set and would provide a more compelling demonstration.

6.3 Failure Mode and Effects Analysis (FMEA) Tables

Item/Step	Failure Mode	Effect of Failure	Severity	Frequency	Cause	D	RPN	Action
C++ Simulation	Speed/Quality of Zoom	Poor User Experience	7	10	High Complexity Code	2	140	Implement Multithreading
Gesture Control	Hand Gesture Recognition	Pinching gesture not recognised for zooming	5	3	Gesture Recognition Library Inaccurate	5	75	Switch to MediaPipe with pretrained models for accurate recognition
Gesture Control Panning	Panning Failure	Unable to pan accurately	8	3	Panning with pointing and corresponding pan values	4	96	Only enable panning via mouse / trackpad
Gesture Control Zooming	Zooming innacurate at different distances from camera	Inability to zoom from slightly further distances	6	3	Taking zoom value directly from Euclidean Distance Calculated	4	72	Normalise Euclidean Distance

Table 4: Table for Software

Item/Step	Failure Mode	Effect of Failure	Severity	Frequency	Cause	D	RPN	Action
IP connection	Maintaining communication with jupyter notebook	Stalled development	8	8	Incorrect configuration on Linux machine	3	192	Disable DHCP configuration
Timing Issues	Stalled when trying to produce a frame	Unable to see view output from hardware implementation	10	9	Too many calculations in a singular clock cycle	5	450	Pipeline and run at lower clock frequencies
Frozen overlay	Frame would not generate for known working bitstreams	Stalled development	8	4	Corrupted kernel	2	64	Reflash PYNQ's SD card

Table 5: Table for Hardware

7 Conclusion

This project successfully designed and implemented a hardware accelerator for the Mandelbrot set on an FPGA, demonstrating the advantages of hardware for computationally intensive tasks. By using fixed point 32 bit arithmetic, the design achieved high precision (Q4.28 format enabling zooms of up to $\sim 1.2 \times 10^6$), which was suitable for the intended purpose of a general Mandelbrot set explorer. Important optimisations included:

- Constraining certain parameters to be powers of 2, allowing for the replacement computationally expensive multiplications and divisions with bit shifts.
- Minimising DSP slice utilisation for squaring operations (reduction from 12 to 10 DSPs). This resulted in more efficient use of the hardware resources.

Additionally, the decision to offload color mapping from hardware to software provided greater flexibility for different colour schemes, allowing us to implement color schemes more suited to color blind people. Performance benchmarking confirmed the expected linear speed-up with an increasing number of parallel engines, demonstrating the effectiveness of parallelisation. The observed time offset of 6.17ms per frame was attributed to various overheads from components in the overlay such as the `packer.v` module (this processes one pixel per cycle, contributing approximately 4.84ms per frame for 720p resolution) and VDMA setup.
Overall, this was a really insightful project which developed our groups skills in hardware & software development. The EEE members of the group were introduced to collaborating on software development using Git and the EIE members gained more experience in developing on actual hardware and addressing the unique issues that come with it. We are proud of our final results as a team as we encountering numerous technical obstacles that required extensive debugging and slowed progress, we successfully delivered a compelling demonstration with a smooth interaction between our hardware and the user interface. The final design effectively provides the user with an educational visualisation of the Mandelbrot set with an interactive, intuitive and responsive user interface.

List of Figures

References

[1] Wolfgang Beyer. *Mandelbrot Set*. Figure. 2005. URL: `https://upload.wikimedia.org/wikipedia/commons/2/21/Mandel_zoom_00_mandelbrot_set.jpg`.

[2] Google. *Gesture recognition guide for Python*. Web Page. 13.1.2025 2025. URL: `https://ai.google.dev/edge/mediapipe/solutions/vision/gesture_recognizer/python?_gl=1*1ppu7l0*_up*MQ..*_ga*MTE4OTQ4OTg2MS4xNzUwMDgyMzMz*_ga_P1DBVKWT6V*czE3NTAwODIzMzMkbzEkZzAkdDE3NTAwODIzNDMkajUwJGwwJGgxMDgwNTE2ODY#code_example`.

[3] npm. *@solid-primitives/mouse README*. Web Page. Apr. 2025. URL: `https://www.npmjs.com/package/@solid-primitives/mouse?activeTab=readme`.

[4] OpenCV. *OpenCV modules*. Web Page. URL: `https://docs.opencv.org/4.x/index.html`.

[5] PaleNeutron. *How to use 'async for' in Python?* Discussion Forum. 2019. URL: `https://stackoverflow.com/questions/56161595/how-to-use-async-for-in-python`.

[6] python. *asyncio — Asynchronous I/O*. Web Page.

[7] tailwindcss. *Adding custom styles*. Web Page. 2025.

[8] Xilinx. *7 Series DSP48E1 Slice User Guide*. Web Page. 27.3.2018 2018. URL: `https://docs.amd.com/v/u/en-US/ug479_7Series_DSP48E1`.

[9] Xilinx. *7 Series FPGAs Memory Resources User Guide*. Web Page. Version 1.14. July 2019. URL: `https://docs.amd.com/v/u/en-US/ug473_7Series_Memory_Resources`.